DG INTERNs HUB

SOC ANALYST INTERNSHIP

WEEK 2

HANDS-ON PROJECT REPORT



Submitted By

Gunjan

Domain: Cybersecurity — SOC Analyst

Tools: Wireshark, Linux, Apache Logs, MITRE ATT&CK, Snort

Submission Date: 15 October 2026

TABLE OF CONTENTS

WEEK 2 — HANDS-ON PROJECT REPORT

S. No.	*Project / Task Title*	*Page No.*
1	*Wireshark PCAP Analysis*	*3*
2	*Network Traffic Capture and Analysis*	—
3	*Ubuntu System Log Analysis*	—
4	*Apache Access Log Analysis*	—
5	*MITRE ATT&CK Technique Mapping*	—
6	*Snort Installation and Rule Testing*	—

Report Sections

- Learning Objectives
- Tools and Technologies Used
- Methodology / Steps Performed
- Findings and Evidence
- Conclusion

TASK 1: WIRESHARK PCAP ANALYSIS

1. Learning Objective

To learn how to inspect a network packet capture using Wireshark and identify source and destination IP addresses, HTTP requests, and DNS queries.

2. Tools Used

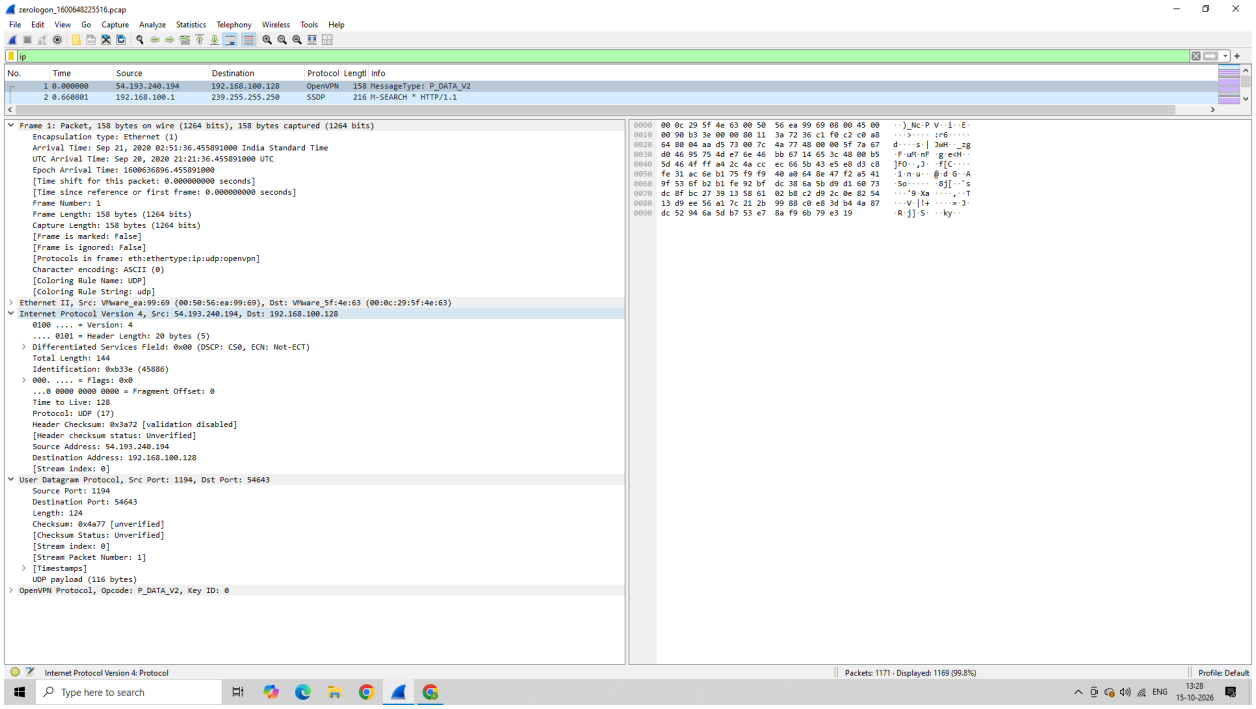
- Wireshark
- Provided PCAP file
- https://drive.google.com/file/d/1UmjEPxhJ-PkGLKniqpnHEhcVwt337k2u/view

3. Steps Performed

1. Opened the provided PCAP file in Wireshark.
2. Applied the `ip` display filter to view IP traffic and identify source and destination addresses.
3. Applied the `dns` display filter to check for DNS queries.
4. Applied the `http` display filter to check for HTTP traffic.
5. Observed the packets displayed by each filter.

4. Findings and Evidence

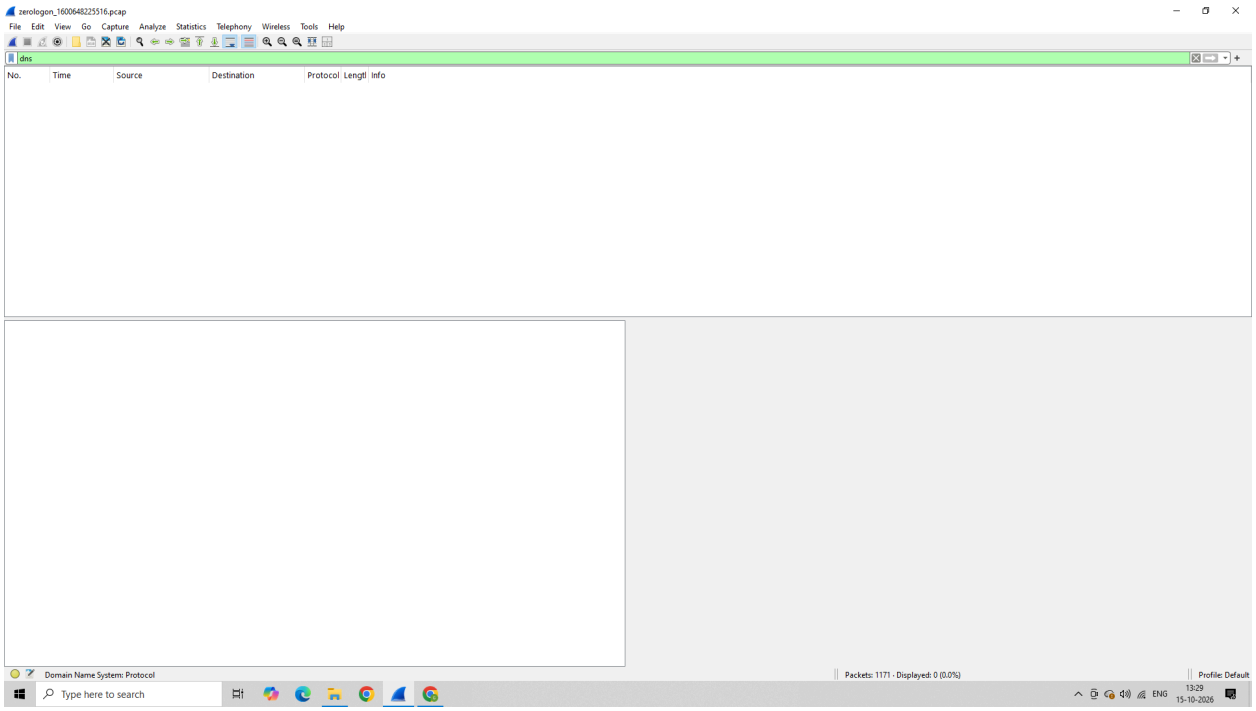
A. IP Traffic Analysis



The `ip` filter displayed IP packets in the capture. The packet list showed source and destination IP addresses.

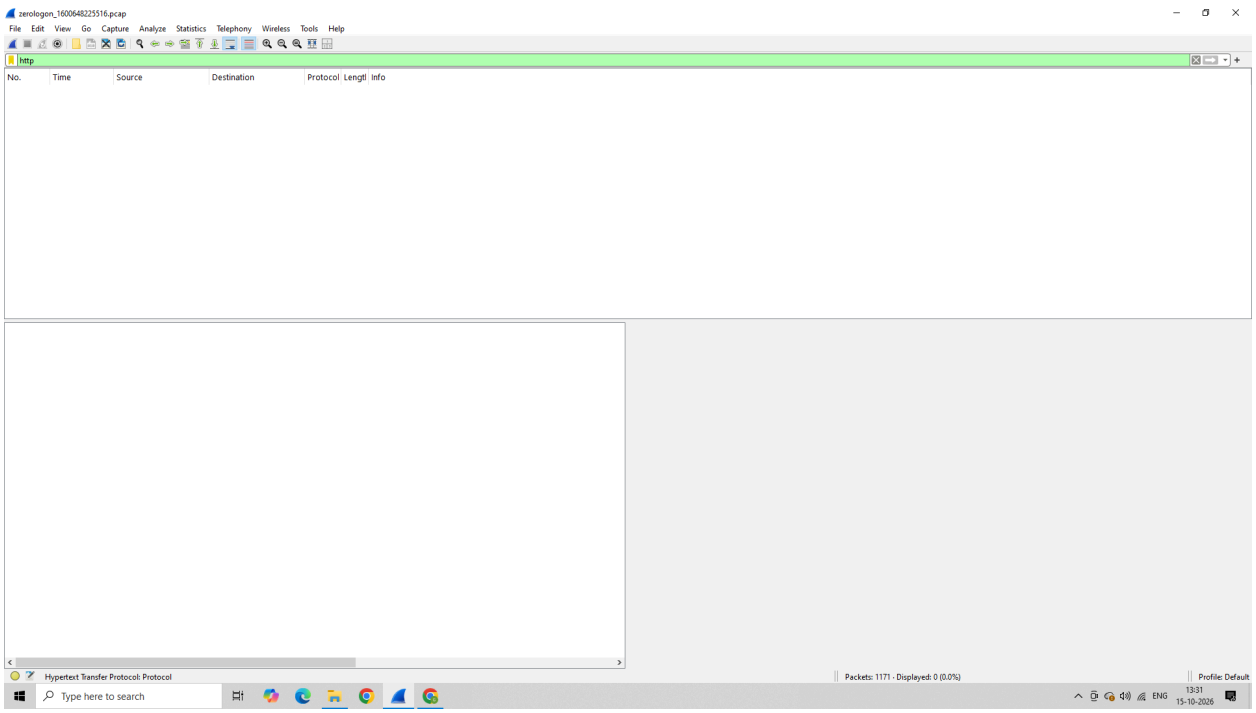
B. DNS Analysis

The `dns` filter displayed zero packets. No DNS packets were visible with this filter in the provided capture.



C. HTTP Analysis

The `http` filter displayed zero packets. No HTTP packets were visible with this filter in the provided capture.



5. Conclusion

In this task, I learned how to open a PCAP file in Wireshark and use display filters to inspect network traffic. I checked IP traffic, DNS queries, and HTTP requests. The DNS and HTTP filters displayed no matching packets in the provided capture. This exercise helped me understand the basic process of network traffic analysis.

TASK 2: OWN NETWORK TRAFFIC CAPTURE

1. Learning Objective

To learn how to capture my own network traffic using Wireshark and observe DNS queries, ICMP packets, and HTTPS-related traffic.

2. Tools Used

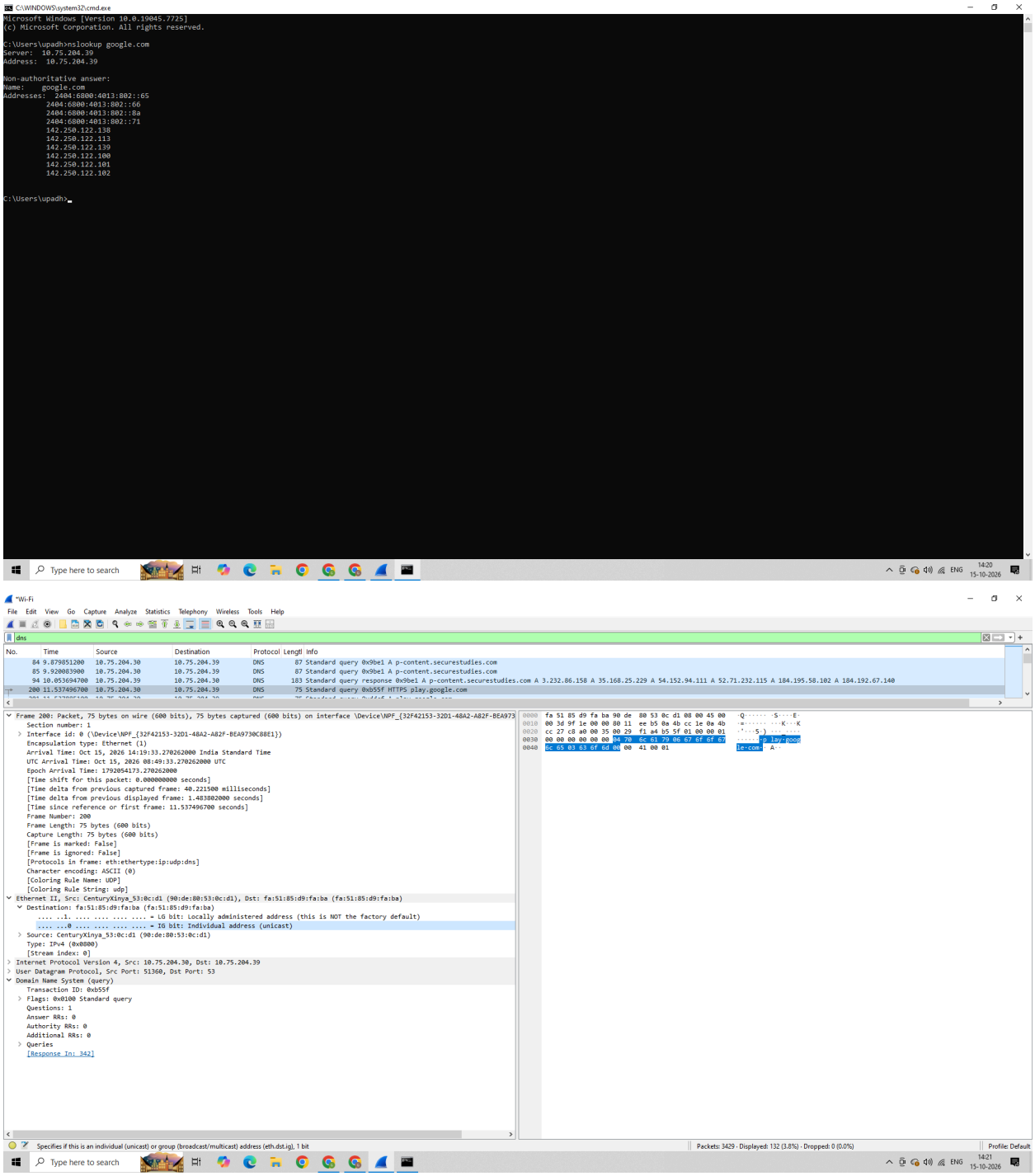
- Wireshark
- Windows Command Prompt
- Web browser
- Sample PDF file

3. Steps Performed

1. Opened Wireshark on Windows and started capturing traffic on the active network interface.
2. Visited Google in the web browser to generate network traffic.
3. Ran `nslookup google.com` in Command Prompt to check DNS resolution.
4. Applied the `dns` display filter to inspect DNS queries and responses.
5. Ran `ping google.com` to test network connectivity.
6. Applied the `icmp` display filter to observe ICMP echo requests and replies.
7. Applied the `tcp.port == 443` display filter to inspect traffic using TCP port 443.
8. Downloaded a sample PDF file while the capture was running.
9. Saved the captured traffic as `Task_2_Network_Traffic.pcapng`.

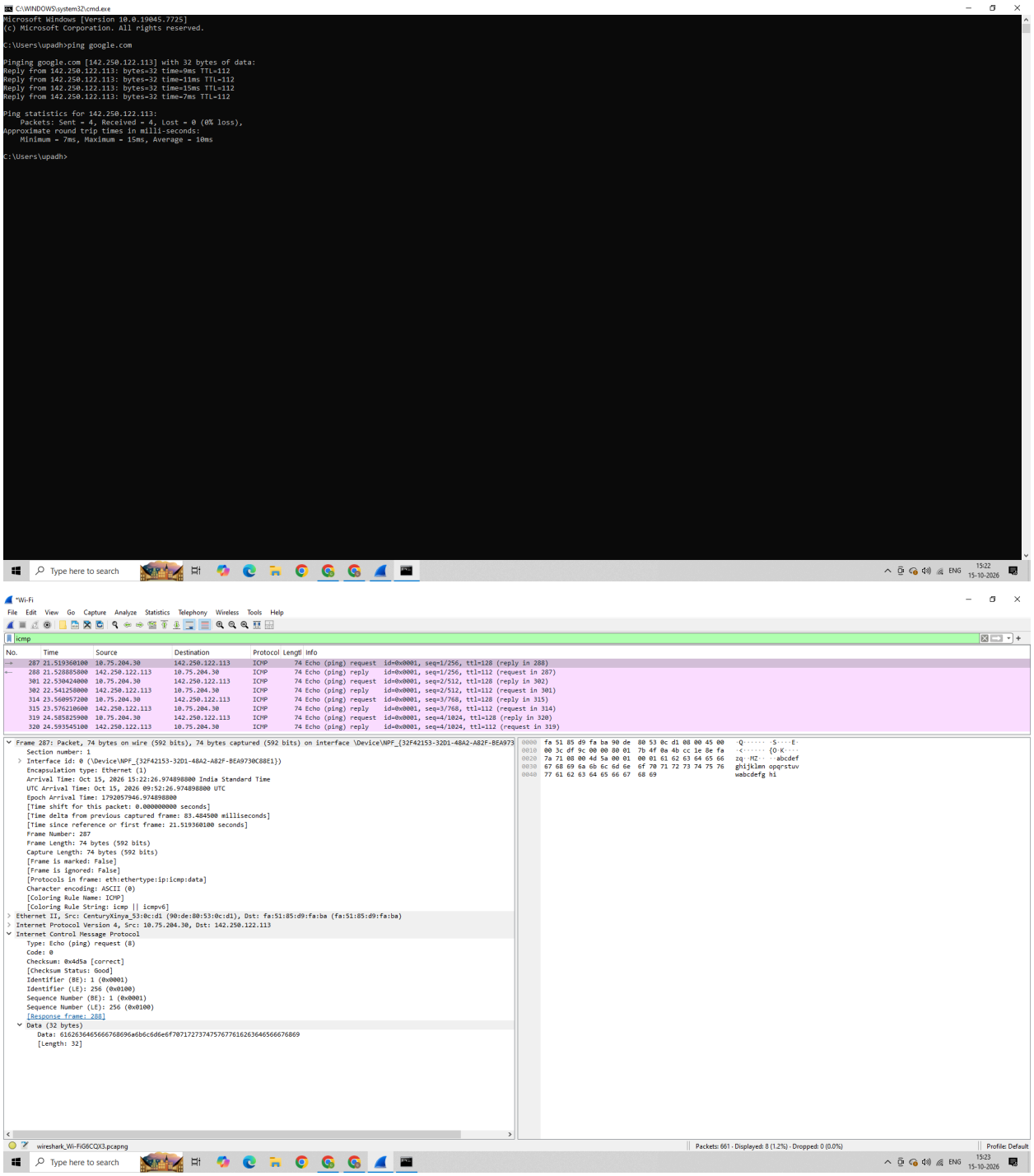
4. Findings and Evidence

A. DNS Traffic Analysis



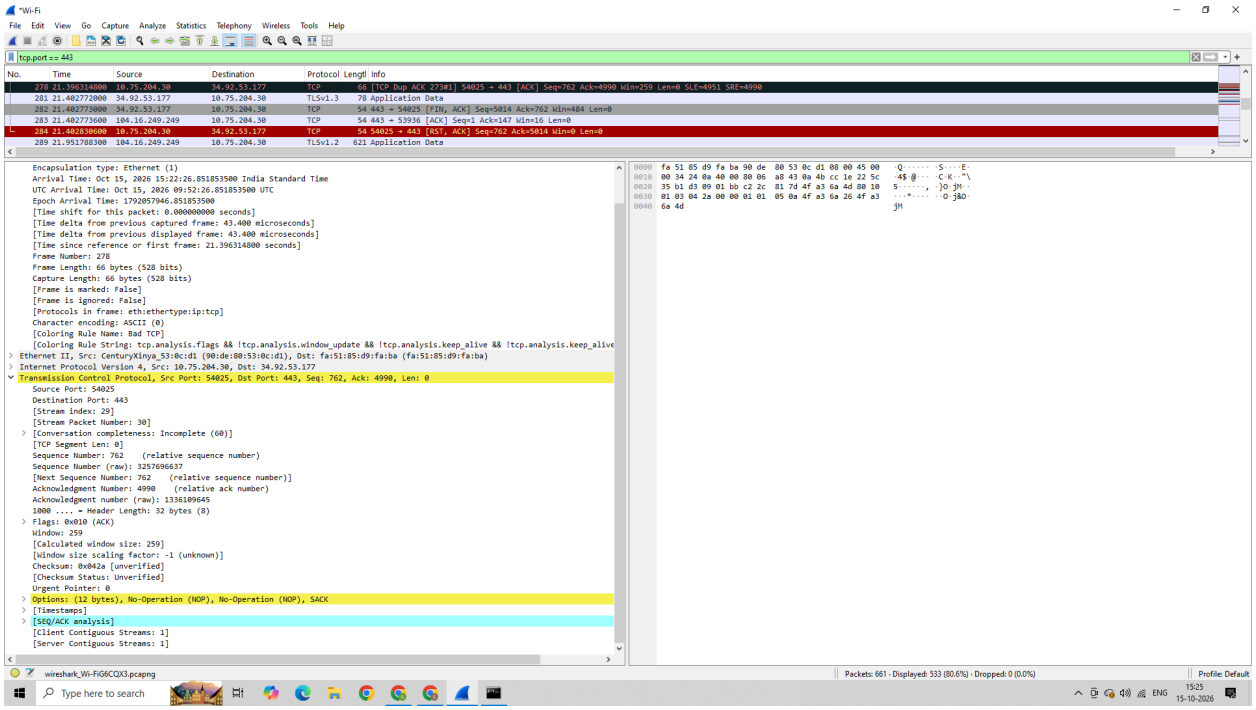
The `dns` filter displayed DNS queries and responses. A query for `play.google.com` was visible in the capture. The `nslookup google.com` command also returned multiple IP addresses for Google.

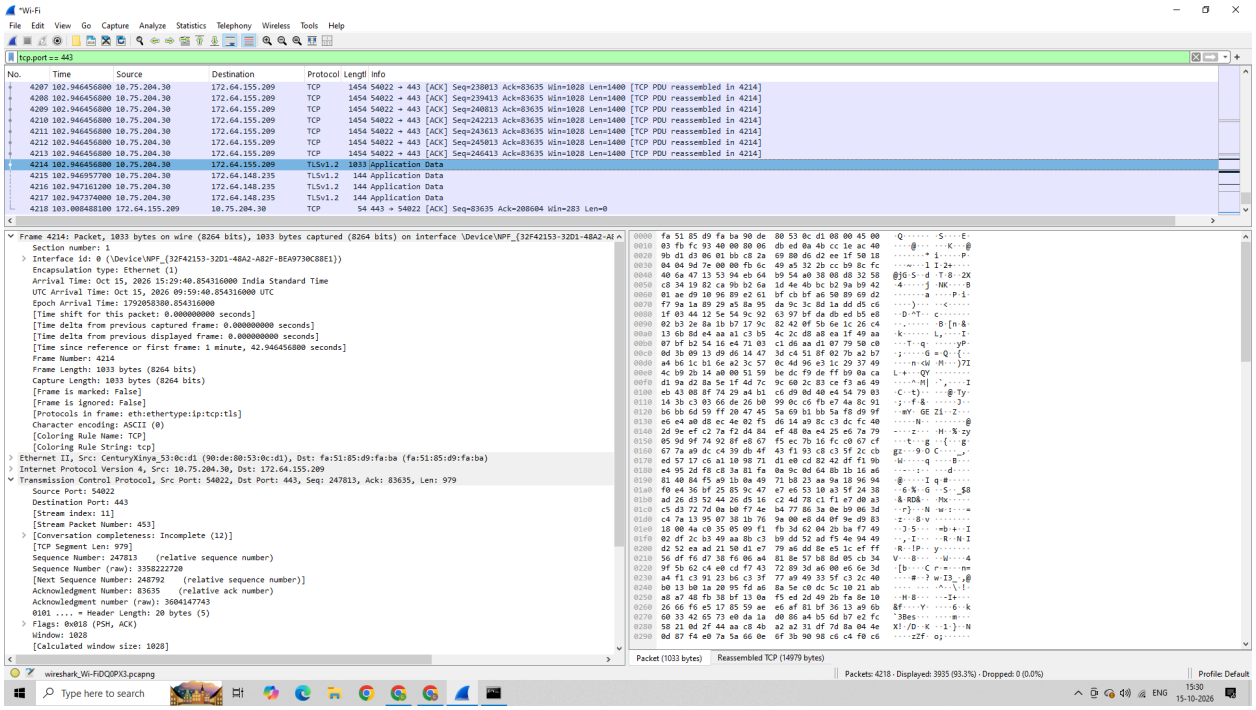
B. ICMP Traffic Analysis



The `ping google.com` command sent four packets and received four replies, with 0% packet loss. Wireshark displayed ICMP echo requests and replies when the `icmp` filter was applied.

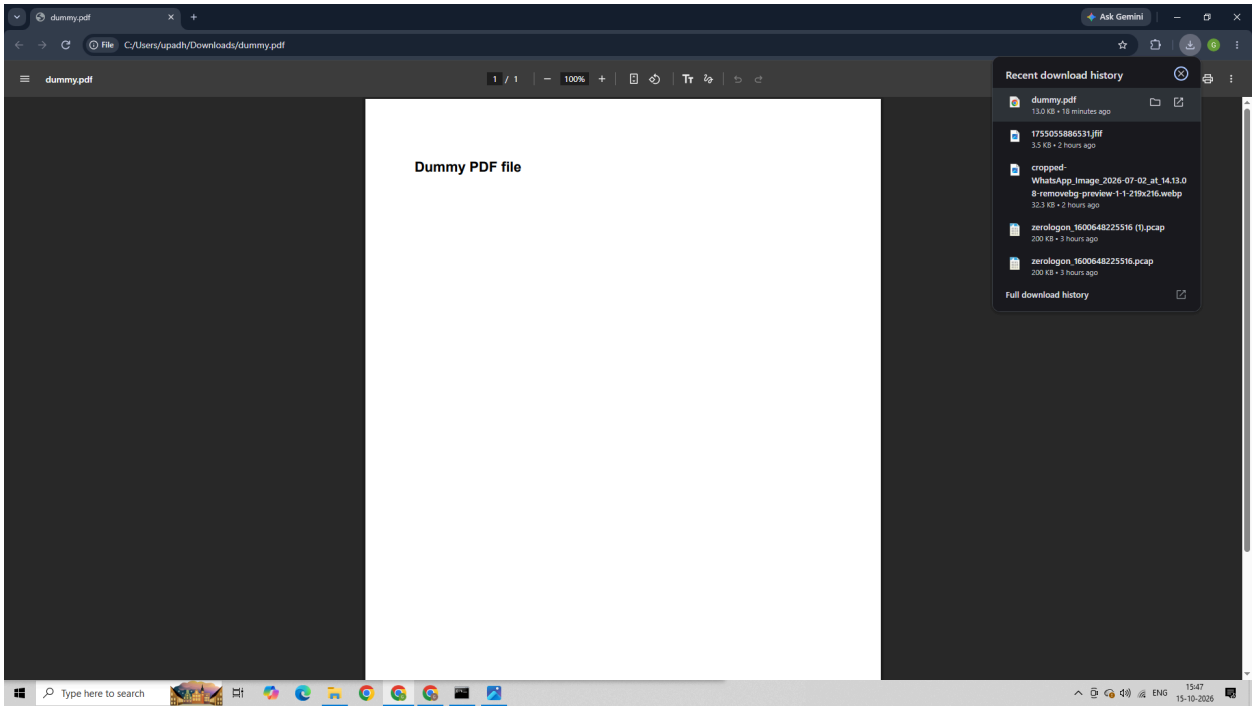
C. HTTPS Traffic Analysis





The `tcp.port == 443` filter displayed TCP traffic using port 443, including TLS application data.

D. File Download Activity



A sample PDF file was downloaded while Wireshark was capturing network traffic to generate additional network activity.

5. Conclusion

In this task, I practised capturing my own network traffic using Wireshark. I used `nslookup` and `ping` to check DNS resolution and network connectivity, and applied display filters to observe DNS, ICMP, and TLS-related traffic. This helped me understand the basic process of network traffic analysis.

TASK 3: LINUX LOG ANALYSIS

1. Learning Objective

To learn how to inspect Linux system logs and identify successful sudo activity using command-line tools.

2. Tools Used

- Kali Linux
- Linux Terminal
- `journalctl`
- `sudo`

3. Steps Performed

1. Checked the Linux distribution using the `/etc/os-release` file.
2. Checked whether `/var/log/auth.log` and `/var/log/syslog` were available.
3. Executed `sudo whoami` to generate a successful sudo activity.
4. Verified the sudo activity using `journalctl` and searched for related log entries.

4. Findings and Evidence

A. Log File Availability

The files `/var/log/auth.log` and `/var/log/syslog` were not available at the expected paths on my Kali Linux system.

B. Successful Sudo Activity

The command `sudo whoami` executed successfully and returned `root`. The system journal showed a sudo entry for user `gunjan`, including the command `/usr/bin/whoami` and the opening and closing of the root session.

C. System Log Review

The `journalctl` command was used to inspect system journal entries. This provided an alternative way to review available system events on the Kali Linux system.

5. Conclusion

In this task, I practised checking Linux log availability and verifying successful sudo activity through the system journal. I learned how `journalctl` can be used to inspect system events when traditional log files are unavailable.

Task 3: Linux Log Investigation

Objective

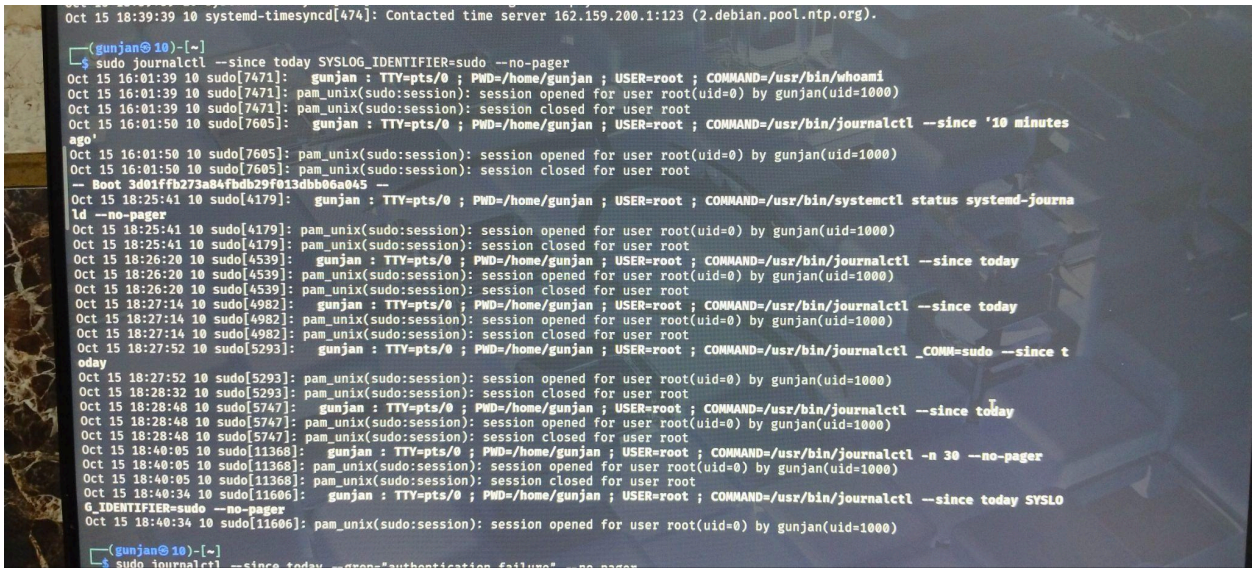
To investigate Linux system logs and identify authentication events, sudo activity, and system errors.

Environment

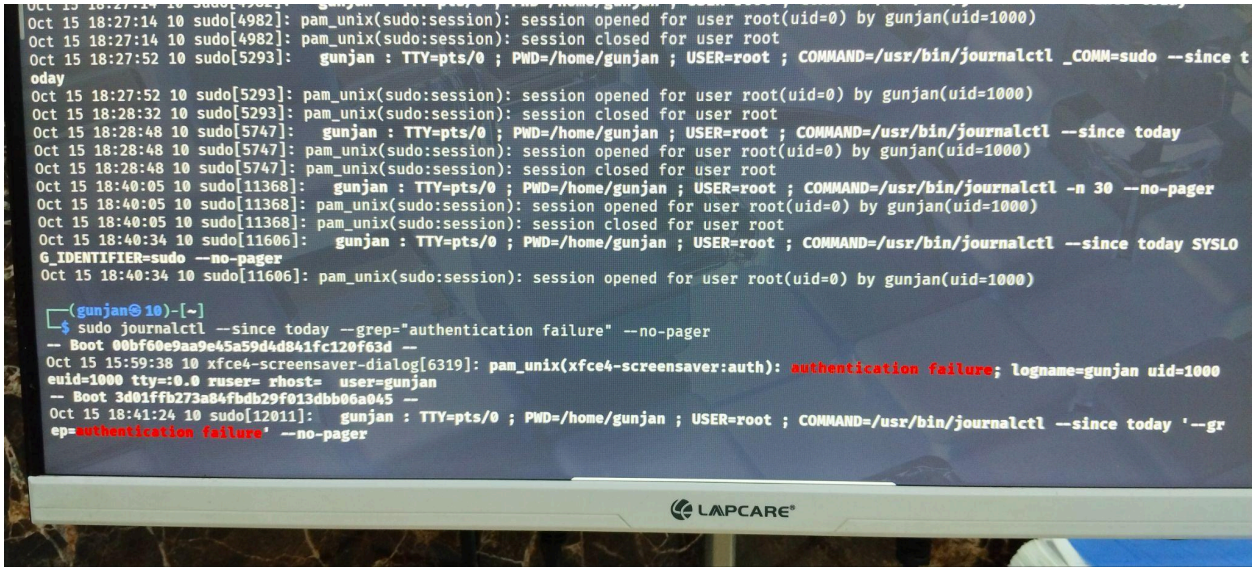
- Operating System: Kali GNU/Linux Rolling 2026.2
- Log source: systemd journal
- Tool: `journalctl`

Findings

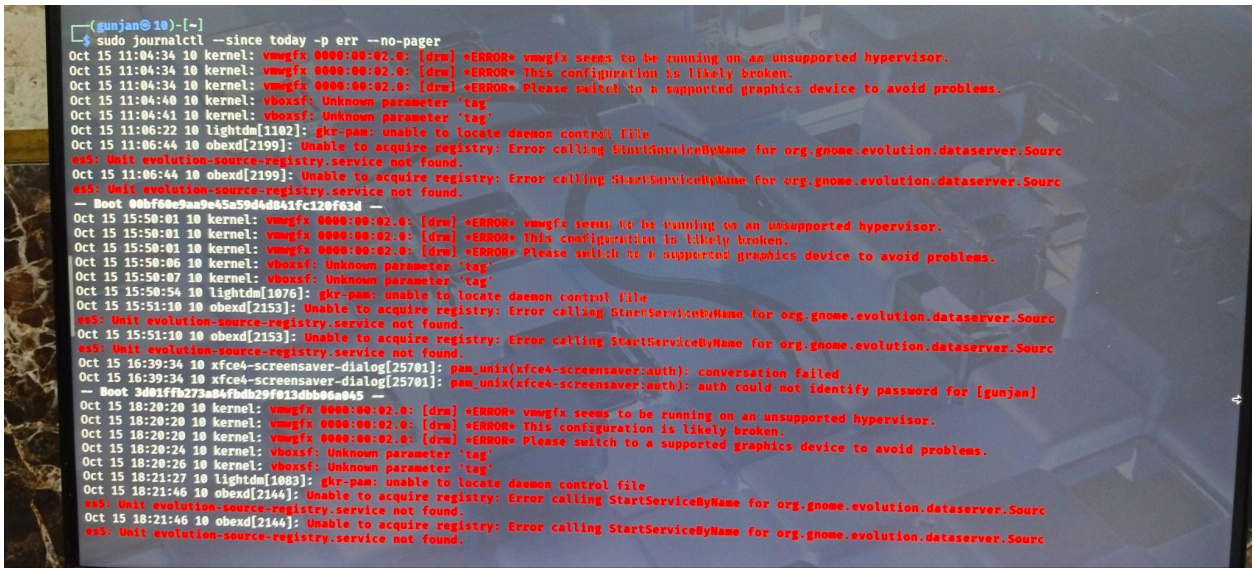
1. **Sudo activity:** The journal recorded sudo sessions opened and closed by the user `gunjan`.



2. **Authentication event:** A screensaver authentication failure was recorded at 15:59:38 on October 15.



3. **System errors:** The journal contained VirtualBox graphics-driver and shared-folder module errors.



4. **Background activity:** Scheduled CRON jobs and system-service events were also recorded.

Conclusion

The journal provided useful information about authentication activity, privileged commands, and system errors. No confirmed cyberattack was established from these entries alone.

Limitation: This investigation was performed on Kali Linux because `/var/log/auth.log` and `/var/log/syslog` were unavailable. The assignment specifies Ubuntu logs, so Ubuntu-based evidence may still be required.

Task 4: Web Server Log Analysis

Objective

To analyze a sample Apache access log and identify the three IP addresses generating the highest number of requests.

Dataset

A publicly available sample Apache access log downloaded from the Elastic examples repository.

Tool Used

Kali Linux Terminal

Methodology

1. Downloaded the sample Apache access log as `access.log`.
2. Used the `head -5 access.log` command to inspect the first five entries.
3. Used an `awk`, `sort`, and `uniq` command pipeline to count requests by IP address and identify the top three.

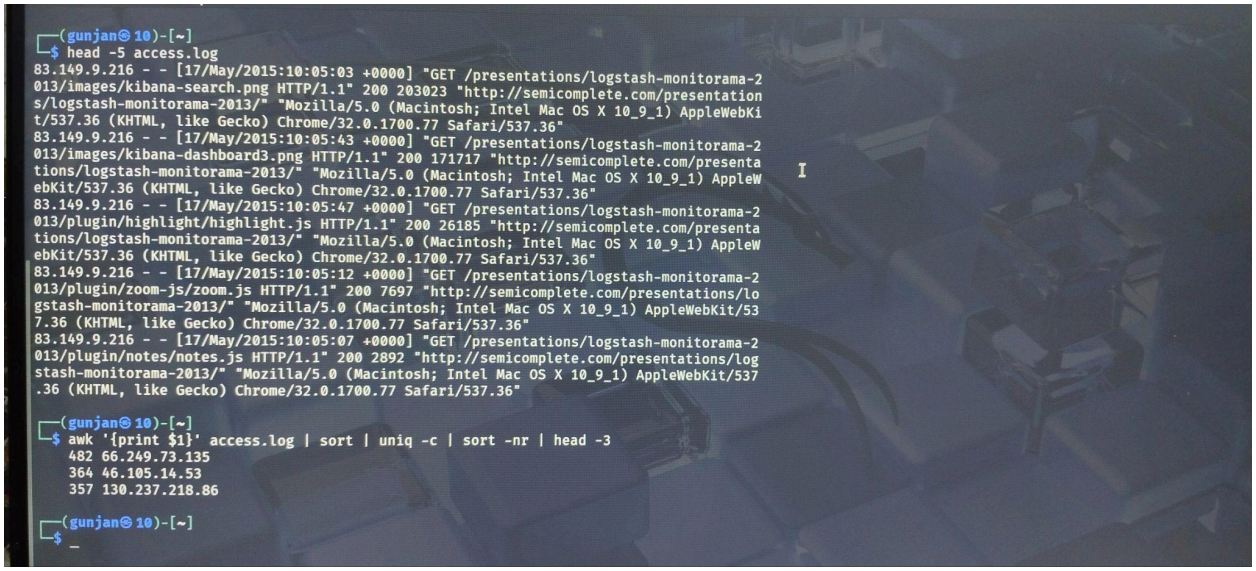
Results

Rank	Source IP Address	Request Count
1	66.249.73.135	482
2	46.105.14.53	364
3	130.237.218.86	357

Findings

The IP address `66.249.73.135` generated the highest number of requests in the analyzed sample, with 482 requests. The other two highest-count IPs were `46.105.14.53` with 364 requests and `130.237.218.86` with 357 requests.

A high request count alone does not prove malicious activity. Further investigation would be required to determine whether the traffic is legitimate or suspicious.



Conclusion

The sample Apache access log was successfully analyzed, and the three IP addresses with the highest request counts were identified. This exercise demonstrated basic web server log analysis using Linux command-line tools.

Task 5: MITRE ATT&CK Mapping

Objective

To understand how common attacker activities map to techniques and tactics in the MITRE ATT&CK framework.

Tool Used

MITRE ATT&CK Official Website: https://attack.mitre.org/

Work Performed

I explored the official MITRE ATT&CK Enterprise knowledge base and mapped three activities: phishing through a malicious link, PowerShell execution, and network service discovery related to port scanning.

1. Phishing

- **Technique:** Spearphishing Link
- **Technique ID:** T1566.002
- **Tactic:** Initial Access
- **Description:** Attackers may send targeted emails containing malicious links to trick victims into visiting harmful websites or downloading malicious files.

2. PowerShell Execution

- **Technique:** Command and Scripting Interpreter: PowerShell
- **Technique ID:** T1059.001
- **Tactic:** Execution
- **Description:** Attackers may abuse PowerShell commands and scripts to execute code, run programs, or perform malicious activities.

3. Port Scanning / Network Service Discovery

- **Technique:** Network Service Discovery
- **Technique ID:** T1046
- **Tactic:** Discovery
- **Description:** Attackers may scan networks and hosts to identify running services and potential targets for further investigation or exploitation.

What I Learned

- How MITRE ATT&CK categorizes attacker behavior into tactics and techniques.
- How phishing can be associated with the Initial Access tactic.
- How PowerShell can be abused to execute commands and scripts.
- How network service discovery can help identify services running on hosts.
- How SOC analysts can use MITRE ATT&CK to describe suspicious activities consistently and support investigations.

Conclusion

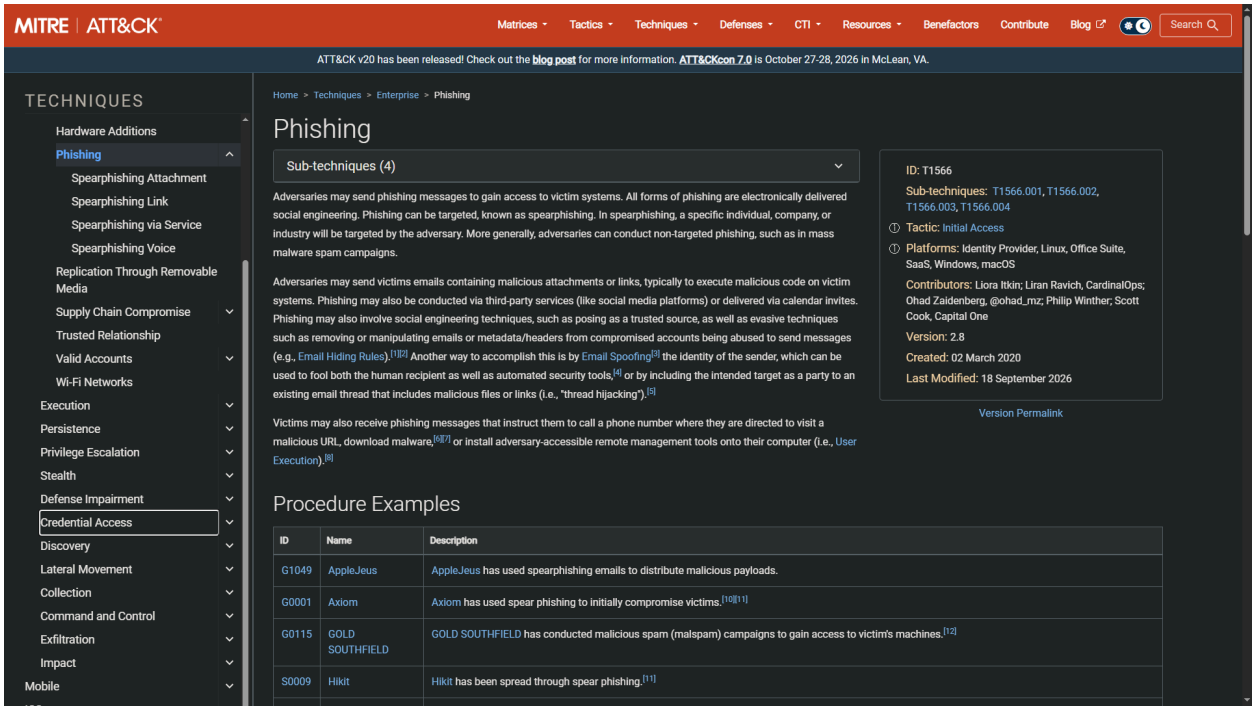
This task improved my understanding of the MITRE ATT&CK framework and how it can be used to classify common cybersecurity activities. I learned to identify relevant technique IDs and tactics for phishing, PowerShell execution, and network service discovery.

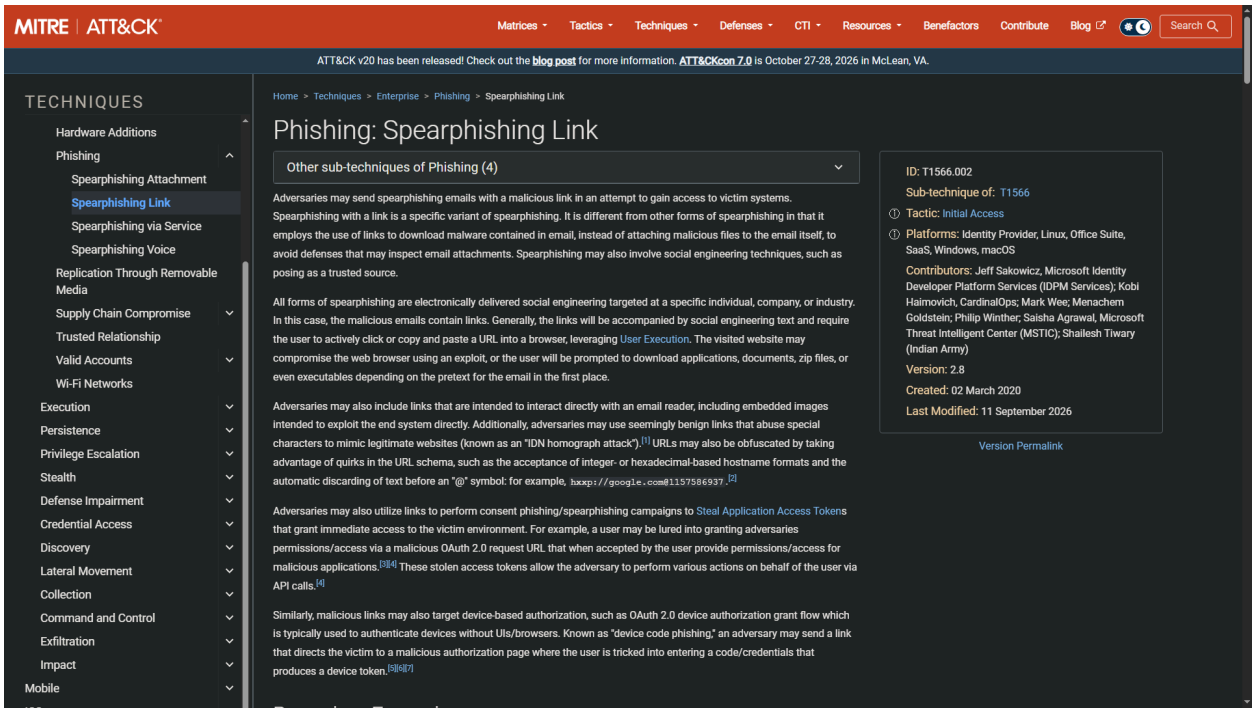
References

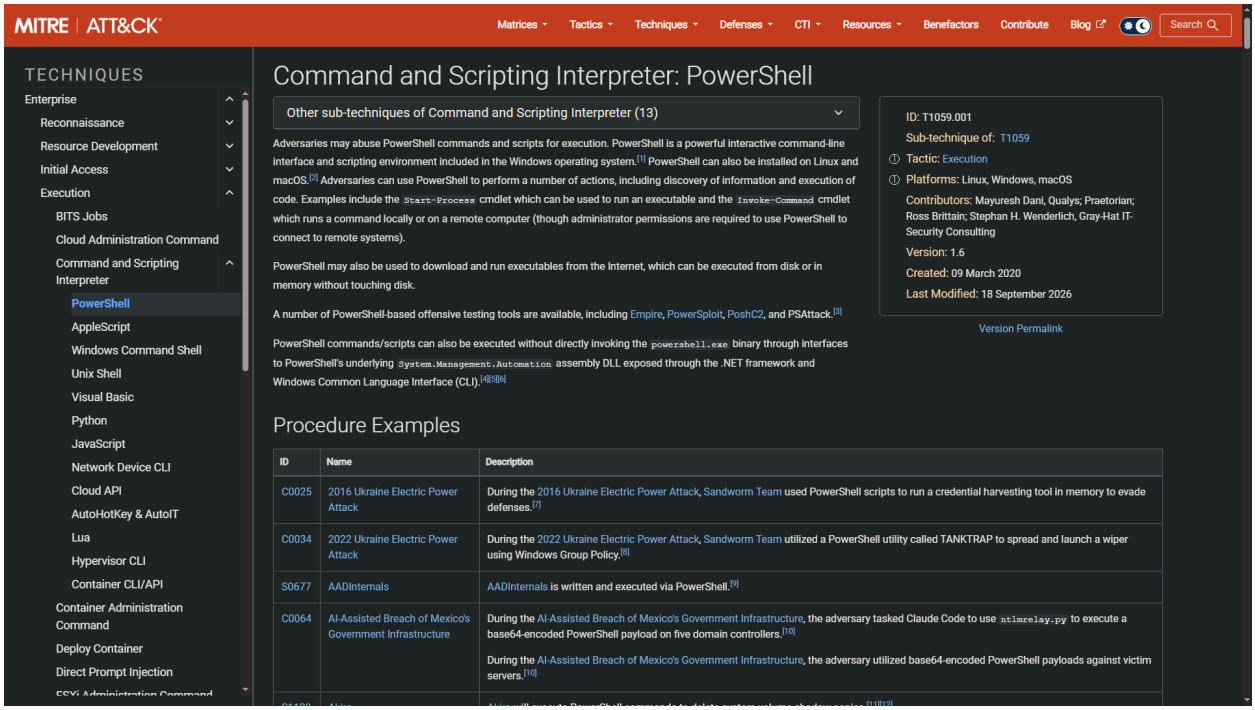
1. Phishing — Spearphishing Link (T1566.002): https://attack.mitre.org/techniques/T1566/002/

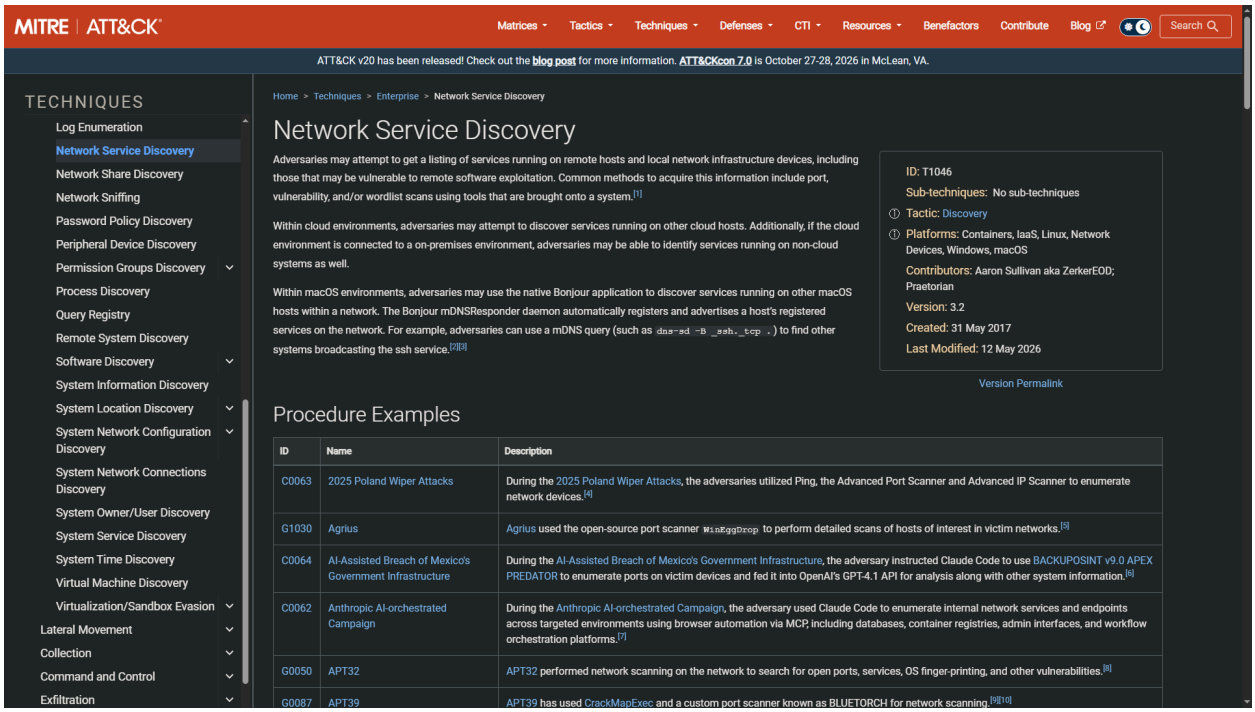
2. PowerShell (T1059.001): https://attack.mitre.org/techniques/T1059/001/
3. Network Service Discovery (T1046): https://attack.mitre.org/techniques/T1046/

Evidence


MITRE | ATT&CK
Matrices
Tactics
Techniques
Defenses
CTI
Resources
Benefactors
Contribute
Blog
Search
ATT&CK v20 has been released! Check out the blog post for more information. ATT&CKcon 7.0 is October 27-28, 2026 in McLean, VA.
TECHNIQUES
Hardware Additions
Phishing
Spearphishing Attachment
Spearphishing Link
Spearphishing via Service
Spearphishing Voice
Replication Through Removable Media
Supply Chain Compromise
Trusted Relationship
Valid Accounts
Wi-Fi Networks
Execution
Persistence
Privilege Escalation
Stealth
Defense Impairment
Credential Access
Discovery
Lateral Movement
Collection
Command and Control
Exfiltration
Impact
Mobile
Home > Techniques > Enterprise > Phishing
Phishing
Sub-techniques (4)
Adversaries may send phishing messages to gain access to victim systems. All forms of phishing are electronically delivered social engineering. Phishing can be targeted, known as spearphishing. In spearphishing, a specific individual, company, or industry will be targeted by the adversary. More generally, adversaries can conduct non-targeted phishing, such as in mass malware spam campaigns.
Adversaries may send victims emails containing malicious attachments or links, typically to execute malicious code on victim systems. Phishing may also be conducted via third-party services (like social media platforms) or delivered via calendar invites. Phishing may also involve social engineering techniques, such as posing as a trusted source, as well as evasive techniques such as removing or manipulating emails or metadata/headers from compromised accounts being abused to send messages (e.g., Email Hiding Rules).[1][2] Another way to accomplish this is by Email Spoofing[3] the identity of the sender, which can be used to fool both the human recipient as well as automated security tools,[4] or by including the intended target as a party to an existing email thread that includes malicious files or links (i.e., "thread hijacking").[5]
Victims may also receive phishing messages that instruct them to call a phone number where they are directed to visit a malicious URL, download malware,[6][7] or install adversary-accessible remote management tools onto their computer (i.e., User Execution).[8]
ID: T1566
Sub-techniques: T1566.001, T1566.002, T1566.003, T1566.004
Tactic: Initial Access
Platforms: Identity Provider, Linux, Office Suite, SaaS, Windows, macOS
Contributors: Liora Itkin; Liran Ravich, CardinalOps; Ohad Zaidenberg, @ohad_mz; Philip Winther; Scott Cook, Capital One
Version: 2.8
Created: 02 March 2020
Last Modified: 18 September 2026
Version Permalink
Procedure Examples
ID
Name
Description
G1049
AppleJeus
AppleJeus has used spearphishing emails to distribute malicious payloads.
G0001
Axiom
Axiom has used spear phishing to initially compromise victims.[10][11]
G0115
GOLD SOUTHFIELD
GOLD SOUTHFIELD has conducted malicious spam (malspam) campaigns to gain access to victim's machines.[12]
S0009
Hikit
Hikit has been spread through spear phishing.[11]


MITRE | ATT&CK
Matrices
Tactics
Techniques
Defenses
CTI
Resources
Benefactors
Contribute
Blog
Search
ATT&CK v20 has been released! Check out the blog post for more information. ATT&CKcon 7.0 is October 27-28, 2026 in McLean, VA.
TECHNIQUES
Hardware Additions
Phishing
Spearphishing Attachment
Spearphishing Link
Spearphishing via Service
Spearphishing Voice
Replication Through Removable Media
Supply Chain Compromise
Trusted Relationship
Valid Accounts
Wi-Fi Networks
Execution
Persistence
Privilege Escalation
Stealth
Defense Impairment
Credential Access
Discovery
Lateral Movement
Collection
Command and Control
Exfiltration
Impact
Mobile
Home > Techniques > Enterprise > Phishing > Spearphishing Link
Phishing: Spearphishing Link
Other sub-techniques of Phishing (4)
Adversaries may send spearphishing emails with a malicious link in an attempt to gain access to victim systems. Spearphishing with a link is a specific variant of spearphishing. It is different from other forms of spearphishing in that it employs the use of links to download malware contained in email, instead of attaching malicious files to the email itself, to avoid defenses that may inspect email attachments. Spearphishing may also involve social engineering techniques, such as posing as a trusted source.
All forms of spearphishing are electronically delivered social engineering targeted at a specific individual, company, or industry. In this case, the malicious emails contain links. Generally, the links will be accompanied by social engineering text and require the user to actively click or copy and paste a URL into a browser, leveraging User Execution. The visited website may compromise the web browser using an exploit, or the user will be prompted to download applications, documents, zip files, or even executables depending on the pretext for the email in the first place.
Adversaries may also include links that are intended to interact directly with an email reader, including embedded images intended to exploit the end system directly. Additionally, adversaries may use seemingly benign links that abuse special characters to mimic legitimate websites (known as an "IDN homograph attack").[1] URLs may also be obfuscated by taking advantage of quirks in the URL schema, such as the acceptance of integer- or hexadecimal-based hostname formats and the automatic discarding of text before an "@" symbol: for example, hxxp://google.com@1157586937.[2]
Adversaries may also utilize links to perform consent phishing/spearphishing campaigns to Steal Application Access Tokens that grant immediate access to the victim environment. For example, a user may be lured into granting adversaries permissions/access via a malicious OAuth 2.0 request URL that when accepted by the user provide permissions/access for malicious applications.[3][4] These stolen access tokens allow the adversary to perform various actions on behalf of the user via API calls.[4]
Similarly, malicious links may also target device-based authorization, such as OAuth 2.0 device authorization grant flow which is typically used to authenticate devices without UIs/browsers. Known as "device code phishing," an adversary may send a link that directs the victim to a malicious authorization page where the user is tricked into entering a code/credentials that produces a device token.[5][6][7]
ID: T1566.002
Sub-technique of: T1566
Tactic: Initial Access
Platforms: Identity Provider, Linux, Office Suite, SaaS, Windows, macOS
Contributors: Jeff Sakowicz, Microsoft Identity Developer Platform Services (IDPM Services); Kobi Haimovich, CardinalOps; Mark Wee; Menachem Goldstein; Philip Winther; Saisha Agrawal, Microsoft Threat Intelligent Center (MSTIC); Shailesh Tiwary (Indian Army)
Version: 2.8
Created: 02 March 2020
Last Modified: 11 September 2026
Version Permalink





Task 6: Basic Snort Setup and Network Traffic Monitoring

1. Objective

The objective of this task was to install Snort, understand its basic working, validate its configuration, and monitor network traffic. I also created a custom ICMP rule to understand how Snort rules can be used to detect specific types of traffic.

2. Tools Used

- Kali Linux
- Snort 3.12.2.0
- Kali Linux terminal
- Snort configuration file (`snort.lua`)
- Local rules file (`local.rules`)

3. Snort Installation and Version Check

First, I installed Snort on my Kali Linux system and checked whether it was working properly. I used the version command to verify the installed version.

The output showed Snort++ version 3.12.2.0. This confirmed that Snort was installed and available to use.

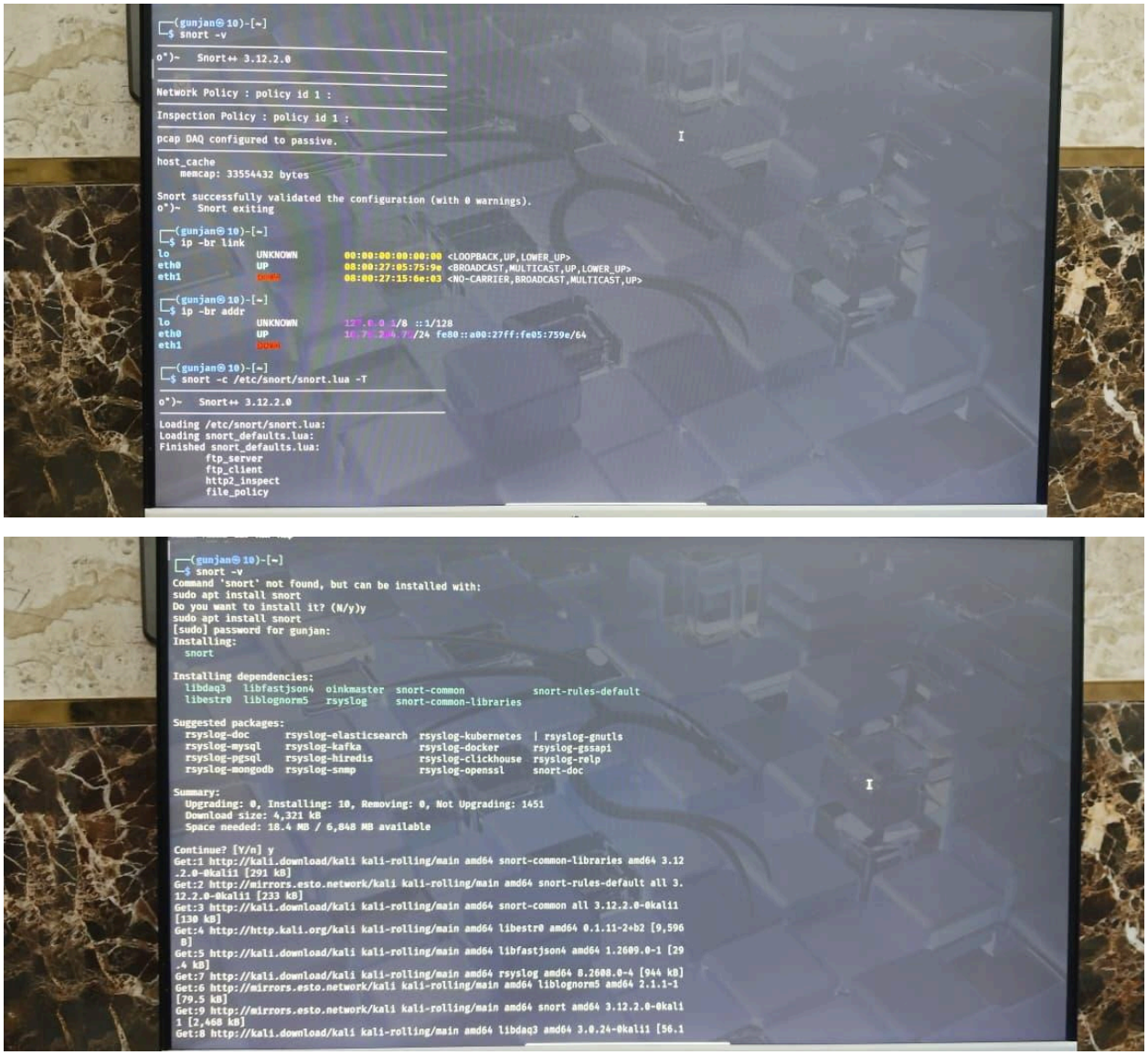


Figure 1: Checking the installed Snort version.

4. Identifying the Network Interface

Before monitoring traffic, I checked the network interfaces available on my Kali Linux system using the `ip -br link` and `ip -br addr` commands.

I found three interfaces: `lo`, `eth0`, and `eth1`. The `eth0` interface was active and had the IP address `10.75.204.71`, so I selected it for the

packet monitoring test. The `lo` interface was used for local communication, while `eth1` was inactive.

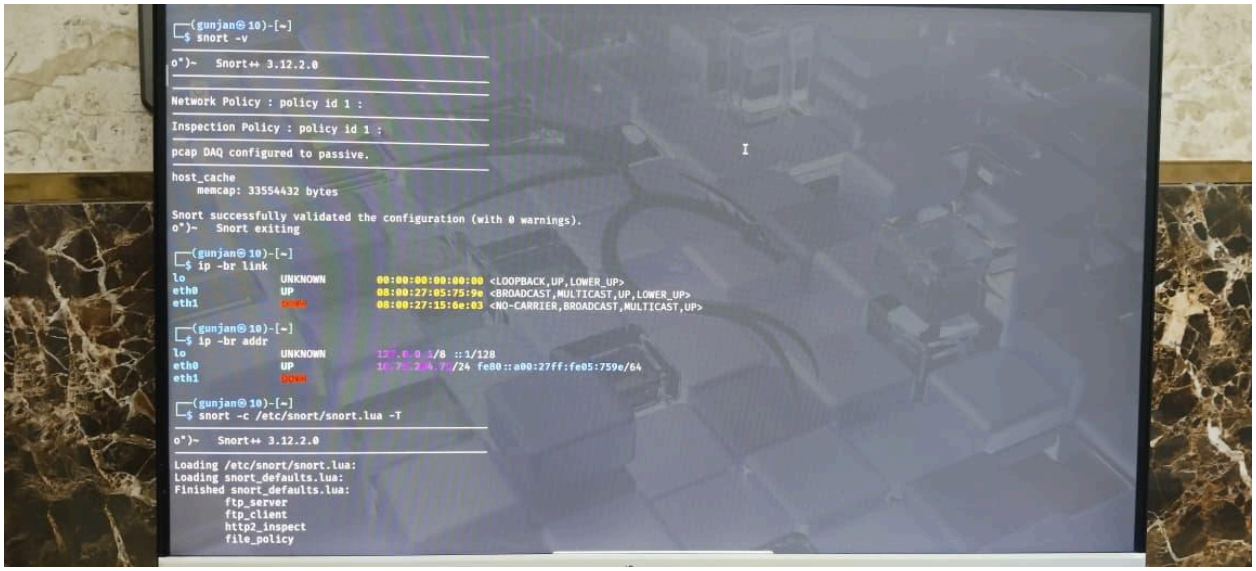


Figure 2: Identifying the active network interface.

5. Testing the Snort Configuration

Next, I tested the Snort configuration using the following command:

```
sudo snort -c /etc/snort/snort.lua -T
```

This command loads the configuration file and checks whether it can be validated without starting continuous monitoring.

The output showed that 219 rules were loaded and the configuration was successfully validated with zero warnings. This confirmed that the existing Snort configuration was working correctly.

6. Monitoring Network Traffic

After validating the configuration, I started Snort on the `eth0` interface using the following command:

```
sudo snort -c /etc/snort/snort.lua -i eth0 -v
```

Here, `-c` specifies the configuration file, `-i eth0` selects the network interface, and `-v` displays packet information in verbose mode.

Snort started packet processing and captured network traffic. After I stopped the monitoring session using Ctrl+C, it displayed the packet statistics.

The output showed that Snort received and analyzed 24 packets, with a total of 2,490 bytes received. The captured traffic included ARP, IPv4, and UDP packets.

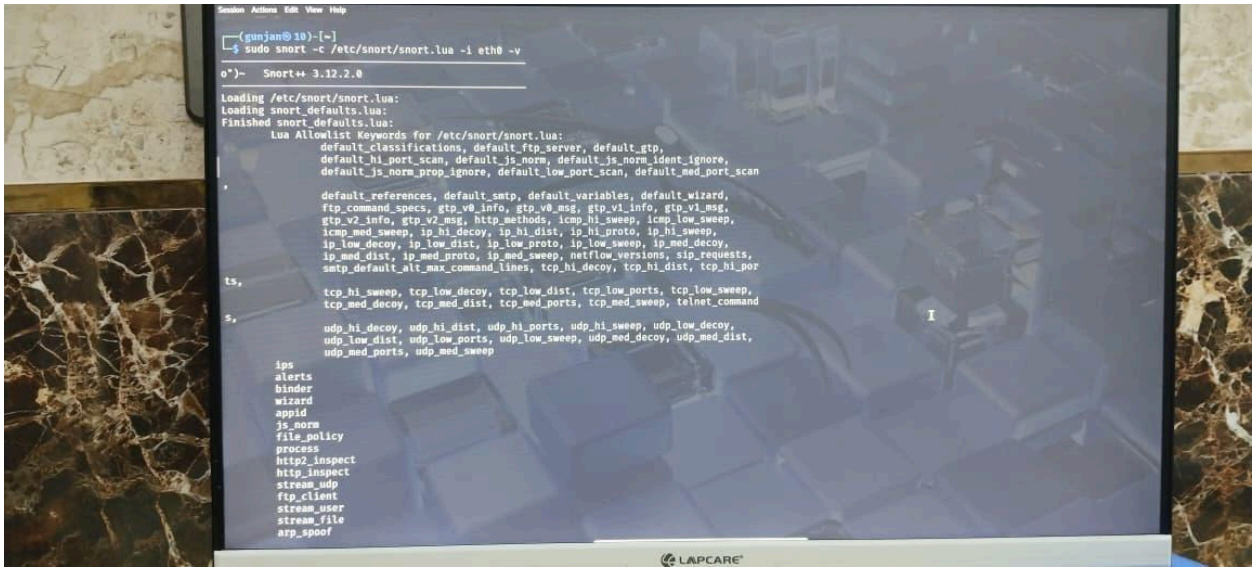


Figure 4: Packet statistics collected during network monitoring.

7. Creating a Custom ICMP Rule

To understand how custom rules work, I added a rule to the `/etc/snort/rules/local.rules` file.

The rule I created was:

```
alert icmp any any -> any any (msg:"ICMP Ping
Detected"; sid:1000001; rev:1;)
```

This rule is intended to generate an alert when Snort processes matching ICMP traffic. ICMP is commonly used by tools such as `ping` to test network connectivity.

The rule was saved in the local rules file. However, I could not confirm that Snort had loaded this custom rule because the configuration test continued to report 219 loaded rules. Therefore, successful alert generation remains unverified.

8. Results and Learning

Through this task, I learned how to:

- Install Snort and verify its version.
- Identify active network interfaces in Linux.
- Test a Snort configuration.
- Monitor network traffic and read packet statistics.
- Create a basic custom ICMP rule.
- Understand the difference between creating a rule and verifying that it generates an alert.

9. Conclusion

This task helped me understand the basic working of Snort as a network intrusion detection tool. I successfully verified the installation, validated the existing configuration, and captured network traffic on the active interface. I also created a custom ICMP rule, although loading and triggering that rule still require further testing.

This practical gave me a better understanding of how network monitoring and detection rules are used in cybersecurity.

